

Protocol-Aware Observability for AI Chat Streaming: A Systematic Review of Tracing and Metrics for Server-Sent Events and WebSocket

Le Huy Tuong (lehuytuong2005@gmail.com)
Le Minh Khoi (leminhkhoid16062k5@gmail.com)
Nguyen Tran Quoc Bao (nguyentranquocbao2501@gmail.com)
Dinh Thi Ngoc Anh (dinhthingocanh030805@gmail.com)
Le Thi Ngoc Han (lengochan090105@gmail.com)

June 2026

Abstract

AI chatbot and retrieval-augmented generation (RAG) systems stream responses token by token so users can see partial output before generation finishes. Developers often default to WebSocket for this, assuming real-time delivery requires a bidirectional channel — but for server-to-client text generation, the traffic is one-way, and Server-Sent Events (SSE) is frequently sufficient and simpler to operate.

Observability is harder for both protocols than for REST. REST tracing works because every request carries HTTP headers; once an SSE or WebSocket connection is open, individual events or messages no longer carry headers, so request-level instrumentation can't see what's happening at the token level.

Rather than treating SSE and WebSocket as competing paradigms, this paper starts from that narrower, practical concern: once a streaming connection exists, how well can we actually observe token-level behaviour on it, and can one instrumentation model do this for both protocols without adding prohibitive overhead? We answer this through a systematic, PRISMA-guided literature review. The review first establishes, as background, that token-level metrics — time to first token (TTFT), inter-token latency (ITL), output throughput — are already well defined in the LLM-serving literature, and that transport-level differences between SSE and WebSocket are minor for one-way token streaming under realistic conditions, a claim we reconcile against literature that seems to say otherwise. What no existing work does, though, is close the tracing gap itself with a single, backend-independent, agent-free instrumentation model spanning both protocols.

From that gap we derive a five-layer protocol-aware observability framework, then validate it on a Spring Boot prototype — driven by both a deterministic mock token generator and a real, locally hosted LLM (Llama 3.2, 3B parameters, served by Ollama), across 80 (mock) and 40 (LLM) repeated runs per configuration. At realistic LLM decode rates, the tracing overhead stays small for both protocols and doesn't diverge in any meaningful way between them ($\leq 7\%$ on time to first token — H1 and H3 hold here). Push the system under transport-bound stress, though, and the picture changes: overhead grows asymmetrically, $+21\%$ for SSE versus $+47\%$ for WebSocket on inter-token latency (H2), pointing to a real cost difference between recording tokens as per-message spans versus lightweight span events on a single stream span.

We also measured on-wire bandwidth directly instead of assuming it — tracing adds almost nothing to SSE’s bytes per token but more than doubles WebSocket’s (+136%), since trace context has to travel inside every message envelope there. Under a simulated weak-network condition (added delay, jitter, 1% packet loss), we couldn’t find a meaningful difference in AI output content loss between the two protocols; both track the configured loss rate within measurement noise.

A second, independent cross-check — an unrelated Python/FastAPI RAG service with no tracing instrumentation at all, driven by a different, faster, cloud-hosted model (Llama 3.1, 8B parameters, served by Groq, 20 runs per configuration) — reproduces transport parity on a completely different stack, and reproduces, more strongly this time, the same real-LLM output-length confound on content-loss measurement seen with Ollama. A fourth, separate experiment validates the framework’s active-connections gauge under 50-way concurrent load for both protocols.

We close by pointing out where the unified model doesn’t apply — a message that needs to anchor its own distributed trace still needs a per-message span, not a span event — and by noting something neither protocol currently solves: a client has no way to detect a silently dropped token. That’s a gap this work surfaces, not one it resolves.

Keywords: Server-Sent Events, WebSocket, observability, distributed tracing, OpenTelemetry, LLM streaming, time to first token, PRISMA.

1. Introduction

1.1 Rationale

Modern chatbot and RAG applications stream responses progressively so that users can read partial output before generation completes. Many developers assume such systems must use WebSocket because it supports bidirectional real-time communication. But for AI text streaming, the dominant pattern is one-way server-to-client delivery of generated tokens, for which SSE can be sufficient and operationally simpler. Both OpenAI and Anthropic expose their streaming APIs over SSE [13, 14].

From an observability standpoint, REST APIs are straightforward to trace: every request carries headers such as `traceparent`. SSE and WebSocket, by contrast, are long-lived connections — headers are present only at connection setup, and subsequent events or messages do not automatically carry HTTP headers [18, 10]. Tracing streaming systems therefore needs additional connection- and message-level context propagation. This is a practical gap: existing tools trace REST and database calls well, but tracing the lifecycle of SSE and WebSocket streams — connection open, per-token delivery, disconnects, cross-service propagation — is less direct.

1.2 Objective and the Central Research Question

This work is guided by one empirically testable research question, informed by a systematic literature review.

Central Research Question. RQ: What gaps exist in message/token-level tracing for long-lived SSE and WebSocket AI streaming connections, and can a single, protocol-independent instrumentation model close them without incurring prohibitive tracing overhead relative to an uninstrumented baseline? It’s a question with a measurable outcome — overhead, and how symmetric it is across protocols — and a clear way to fail: overhead that turns out large, or asymmetric between protocols in a way that would undercut the “unified” claim.

Supporting literature context. Answering the RQ first needs the review to nail down three background points: what metrics people commonly use to compare SSE and WebSocket; when SSE is sufficient for AI token streaming and when WebSocket earns its extra complexity; and how well

current observability tools already support REST tracing compared to SSE/WebSocket tracing. None of this needs new measurement on its own — it is synthesised from existing sources (§4.3) and feeds directly into the gap analysis behind the framework’s design (§6.1) and the hypotheses tested experimentally below.

1.3 Hypotheses

We break the central RQ down into three hypotheses that can each fail on their own terms, tested experimentally in §6.4.

The first, H1, is that overhead is acceptable: unified instrumentation should add only negligible tracing overhead — relative to an uninstrumented baseline — once you’re at LLM-realistic token generation rates. The second, H2, is about a design choice: recording each token as a lightweight event on a per-stream span should cost less than giving each message its own span, and that gap should widen as token or message frequency climbs. The third, H3, is protocol independence itself: SSE and WebSocket should show statistically indistinguishable baseline tracing cost under the same instrumentation model, at least when neither protocol’s own transport mechanics is the bottleneck.

H1 and H3 fail if overhead turns out large, or systematically asymmetric between protocols, at realistic rates. H2 fails if per-message and per-event designs end up costing about the same under stress — which would undercut the whole case for choosing span events over per-message spans in §6.1.

1.4 Contributions

What this paper actually contributes is one thing: a protocol-independent instrumentation model for message- and token-level observability of SSE and WebSocket AI streaming. Concretely, that’s a two-tier design — one connection-lifetime span, plus lightweight per-token span events — derived from the gap analysis in the literature review, not asserted from first principles. Everything else in the paper builds the case for that one model: the review (§4.3) establishes the gap it closes; the five-layer framework (§6.1) works out its structure; the Java/Spring Boot prototype and its test of three falsifiable hypotheses (H1–H3, §6.4) check whether the model actually achieves protocol independence without prohibitive overhead; and the real-LLM (Ollama, Groq) and concurrent-load cross-checks push that test beyond a single synthetic benchmark.

2. Background

2.1 REST API Observability

REST observability focuses on request latency, status code, throughput, error rate, and payload size, with distributed tracing achieved by propagating trace context through HTTP headers. Each service extracts incoming context, creates a span, and propagates context downstream. This fits short-lived request–response communication [18, 20].

2.2 Server-Sent Events

SSE pushes events from server to client over a single long-lived HTTP connection, using the `text/event-stream` media type and the EventSource API [17]. It suits one-way streaming — notifications, dashboards, AI text. After the connection is established, individual events carry no independent HTTP headers, so per-event tracing needs server-side metadata or payload-embedded identifiers [9].

2.3 WebSocket

WebSocket provides a persistent, bidirectional channel established via an HTTP upgrade handshake, after which communication uses WebSocket frames [16]. Post-handshake frames don't behave like REST requests. So tracing WebSocket needs two levels of instrumentation — connection-level and message-level — with the latter relying on application metadata such as a trace ID embedded in the message envelope [11, 12].

2.4 Distributed Tracing and Trace Context

Distributed tracing tracks how an operation moves across services as a tree of spans [20]. The W3C Trace Context specification standardises `traceparent`/`tracestate` headers [18], and OpenTelemetry provides a vendor-neutral ecosystem [19]. The standard model works most naturally for HTTP request-response. Streaming needs additional design, because post-handshake messages may not carry headers.

2.5 AI/LLM Token Streaming

LLM serving is characterised by token-level latency metrics: time to first token (TTFT), the time until the first token appears; inter-token latency (ITL, also TPOT), the average gap between consecutive tokens; and output throughput in tokens per second [6, 7]. Interactive applications benefit most from low TTFT and ITL. In RAG, TTFT additionally includes retrieval time before generation begins.

3. Methods

This review follows the PRISMA 2020 reporting guideline [1] as a lightweight protocol. As an early-stage review, final inclusion counts are provisional pending full-text assessment.

Sources were drawn from academic databases (IEEE Xplore, ACM Digital Library, Springer-Link, arXiv, Google Scholar) and authoritative technical sources (OpenTelemetry, Spring, MDN, W3C, IETF, vendor engineering blogs), last searched 23 June 2026, using query clusters combining SSE/WebSocket terms with performance, observability/tracing, LLM/AI-streaming, and OpenTelemetry/distributed-tracing terms. Inclusion criteria: sources discussing SSE, WebSocket, HTTP streaming, or long-lived connections with latency/throughput/bandwidth/connection metrics, tracing/telemetry/monitoring, or Java/Spring Boot/OpenTelemetry implementation detail. Exclusion criteria: frontend-only, no metric/observability content, promotional, off-topic, or outdated (unless foundational).

Records were screened by title/abstract, then full text, extracting protocol, system context, metrics reported, observability method, trace granularity, strength, and limitation per source. Sources range from peer-reviewed conference/journal papers to official standards and specifications to engineering blogs and vendor documentation. Because the topic is young and partly documented in industry channels rather than academic venues, we note explicitly, wherever it matters for a claim, whether the underlying source is peer-reviewed research or an industry/vendor write-up, rather than treating all sources as equally rigorous. Given how heterogeneous the sources are, we use narrative (qualitative) synthesis, grouping evidence by protocol and by metric layer.

4. Results

4.1 Study Selection (PRISMA Flow)

Figure 1 summarises identification and screening. Counts are provisional: identification reflects the corpus gathered on the search date; final inclusion is to be confirmed after full-text assessment.

Identification. Academic DBs ≈ 28 ; standards/docs ≈ 12 ; engineering blogs/repos ≈ 15 . Total ≈ 55 (incl. 23 from a prior baseline review). After duplicate removal: ≈ 48 .

Screened (title/abstract): ≈ 48 ; excluded (frontend-only, marketing, off-topic) ≈ 13 .

Full-text assessed: ≈ 35 ; excluded (no metric/observability, outdated) ≈ 10 [to finalise].

Included in synthesis: ≈ 25 [provisional].

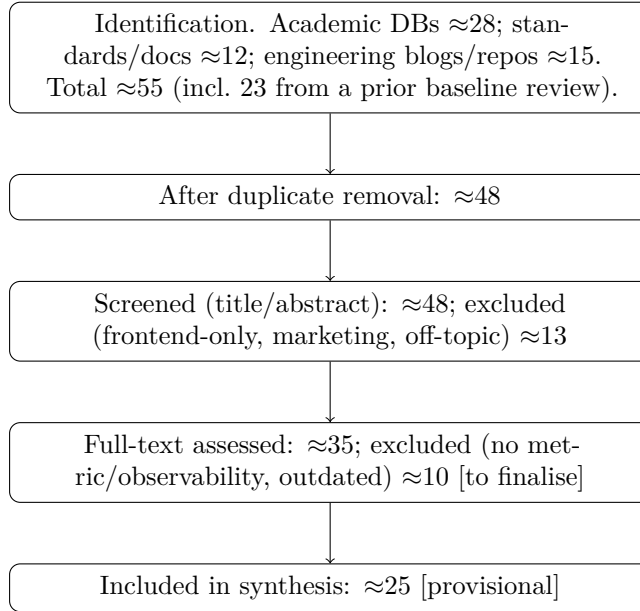


Figure 1: PRISMA 2020 flow (provisional working counts).

4.2 Study Characteristics

Table 1 lists representative included sources.

4.3 Literature Synthesis

The synthesis below covers the three pieces of supporting context introduced in §1.2 — metrics, transport choice, tool support — then arrives at the tracing gap that is this paper’s actual research question.

Metrics. Two distinct layers emerge. Transport-level (from academic measurement studies): latency, throughput, bandwidth, CPU/memory, TCP connection count, and message frequency [3, 4, 2]. Token-level (LLM serving): TTFT, ITL/TPOT, tokens/s, and end-to-end latency [6, 7]. The token-level set, not the REST set, is the relevant comparison basis for AI streaming.

SSE vs. WebSocket in practice. For one-way server-to-client AI text streaming, SSE is sufficient and simpler — automatic reconnection, CDN compatibility, no sticky sessions — and is the industry convention: OpenAI, Anthropic, and other major LLM providers stream over SSE [13, 14]. This reflects current practice rather than peer-reviewed measurement: no peer-reviewed source was found

Table 1: Representative included sources.

ID	Year	Focus	Metrics/contribution
WWW’21 WebSocket adoption [2]	2021	WS in the wild (71k connections)	adoption, usage patterns
WS full-duplex eval [3]	2014	WS vs TCP	latency, network traffic
WS data transmission [4]	2019	WS RTC	throughput, latency
WS benchmark infra [5]	2016	server-side benchmark	black-box throughput
LLM serving eval [6]	2025	LLM serving	TTFT, ITL, TPS, pitfalls
Chiron autoscaling [7]	2025	LLM serving	TTFT, TBT/ITL defs
OTel GenAI semconv [8]	2026	LLM observability	token-usage attributes
OneUptime SSE [9]	2026	SSE tracing (industry, not peer-reviewed)	connection + event span
OneUptime WS [10]	2026	WS tracing (industry, not peer-reviewed)	cardinality problem (unverified in peer-reviewed literature)
Alibaba LoongSuite [11]	2026	WS observability (industry, not peer-reviewed)	chunk metrics, agent
SSE vs WS for LLM [13]	2026	transport choice (industry claim, not peer-reviewed)	latency negligible

benchmarking SSE against WebSocket specifically for LLM token streaming. So we don’t take the “negligible difference” claim on faith. Section 6.4 tests it directly (H3) on our own prototype, and finds baseline inter-token latency statistically indistinguishable between SSE and WebSocket at realistic decode rates (20–50ms/token).

WebSocket is preferable when mid-stream bidirectional input is required — interruption, voice, collaboration — again per industry sources rather than an academic benchmark [10, 15]. Section 5.1 works through a real tension here: the industry claim of a negligible difference doesn’t sit well with general-purpose academic transport literature, which reports larger SSE/WS gaps under other conditions. We resolve it there with our own experimental evidence rather than picking a side between the two industry claims.

Tool support. REST tracing is mature. LLM calls now have an emerging standard via the OpenTelemetry GenAI semantic conventions [8], but most attributes are experimental and don’t yet cover the streaming lifecycle. SSE and WebSocket tracing still need deliberate, manual instrumentation [9, 10, 11].

The tracing gap: this paper’s research question. Against that background, two recurring gaps define the RQ this paper actually answers. First, after the handshake, frames and events carry no HTTP headers, breaking REST-style propagation. Second, emitting one span per message or per token is reported to explode span cardinality and overwhelm the collector — a claim we found only in vendor engineering practice [10], not in any peer-reviewed source. No peer-reviewed work addresses span-cardinality management for message/token-level streaming traces; that absence is itself a gap this review surfaces.

The SSE pattern [9] and the WebSocket pattern [10, 11] independently converge on the same two-tier shape to work around these gaps. Still, no source offers a single, backend-independent, agent-free model that captures token-level metrics across both protocols — the gap this work targets. Both recurring gaps motivate the two-tier model directly: a connection-lifetime span plus a lightweight per-event/token record (design rationale in §6.1).

We don’t rest the second gap on vendor claims alone. Our prototype gives it direct empirical support, tested as H2: under transport-bound stress (§6.4), a WebSocket adapter that emits one child span per message costs measurably more (+47% inter-token latency) than an SSE adapter recording tokens as span events on a single stream span (+21%). Per-message span emission simply doesn’t scale as well as event-level recording as message frequency grows. Section 6.4 tests H1–H3 on the resulting model overall: the same instrumentation applied to both protocols yields near-identical relative tracing overhead at realistic token rates (H1, H3 supported), though the underlying per-message vs. per-event span mechanics diverge under stress (H2 supported).

5. Gap Analysis

REST observability is mature because each request carries headers. SSE and WebSocket lack header-based per-message propagation, and risk a cardinality blow-up under naive per-message spans. The synthesis points to a unified two-tier model — one stream span anchoring per-event/token records, applied identically to SSE and WebSocket, and exportable without a mandatory backend — that would close the gap left by both agent-based tooling and protocol-specific tutorials.

5.1 A Contradiction in the Literature, and Its Resolution

The industry-practice sources on SSE vs. WebSocket in AI streaming (§4.3) don’t agree with the general transport-performance academic literature, and the disagreement is worth taking seriously

rather than waving off as noise — it’s a disagreement between peer-reviewed academic measurement and industry/practitioner claims, not between two peer-reviewed claims.

General-purpose measurement studies of SSE and WebSocket — WWW’21’s large-scale in-the-wild WebSocket study [2], IEEE full-duplex evaluations [3, 4], and mobile-network SSE/WS comparisons under 3G/4G — report transport differences that are not negligible under constrained bandwidth, high message frequency, or adverse network conditions. Practitioner sources specific to LLM streaming [13, 14], by contrast, report the difference as negligible ($\approx 5\text{--}7\text{ms}$) for AI token generation, without peer-reviewed measurement behind that number. Taken at face value, these two claims conflict: is the transport difference material or not?

Our own way into this tension was simpler than the resolution above makes it sound: rather than starting from the academic-vs-industry framing, our first instinct was just to put both protocols under the best possible network condition and see whether the industry claim of a negligible difference actually held up there. If it did, that would already hint that the academic literature’s larger reported gaps came from testing under harsher conditions, not from some inherent difference between the protocols themselves.

Interestingly, the conflict dissolves once the workload regime is made explicit — something none of the LLM-streaming sources state precisely. The general transport literature measures difference under high-frequency or network-constrained transmission, where the transport itself is the bottleneck. LLM token generation, at $\approx 10\text{--}50\text{ms}$ per token, is far slower than either protocol’s per-frame overhead on a healthy connection, so the transport usually isn’t the bottleneck. That’s exactly the boundary condition the practitioner sources implicitly assume but don’t test.

Our own experiment (§6.4) reproduces this resolution directly instead of merely asserting it: the SSE/WS difference is statistically negligible at 20–50ms/token (matching the LLM-streaming claim) but reappears under 0ms/token transport-bound stress (matching the general transport literature), on the same prototype, with the same instrumentation. The apparent contradiction in the literature isn’t a disagreement about SSE or WebSocket as such — it’s a difference in which regime each body of work operates in, a distinction the practitioner sources don’t draw explicitly and this paper makes precise and testable.

6. Proposed Protocol-Aware Observability Framework

6.1 Design Rationale

The five layers in Table 2 aren’t an assumed starting point. Each is a direct consequence of a specific finding from the literature synthesis (§4.3), and the count of five reflects the number of qualitatively distinct concerns those findings identify.

- L1 (REST/API) is inherited, not novel: the synthesis found REST tracing already mature via header-based context propagation. The framework reuses this maturity as its foundation rather than re-deriving it.
- L2 (DB/downstream) extends the same header-propagation model one hop further, to the database and outbound calls a request makes before or after a stream begins (e.g. retrieval in a RAG pipeline). It’s required for end-to-end traces, but like L1, isn’t where the review identified a gap.
- L3 (Streaming connection) exists because of the tracing gap’s first component: once a connection is established, frames and events carry no HTTP headers, so REST-style per-request propagation

can’t describe a stream’s lifecycle. L3 is the minimal layer needed to restore lifecycle visibility — open, duration, disconnect, active count — without assuming headers survive the handshake.

- L4 (Message/event) exists because of the tracing gap’s second component: naively creating one span per message or token, symmetric with L1’s per-request span, explodes cardinality and overwhelms the collector. L4 is deliberately not a per-message span layer; it’s the lightweight span-event record that the two-tier model anchors under the L3 connection span.
- L5 (AI streaming) exists because the metrics synthesis identified token-level metrics (TTFT, ITL, throughput) as a metric family distinct from both generic transport metrics (L1–L3) and generic message metrics (L4): they characterise generation quality, not connection health, and a RAG system’s TTFT further decomposes into retrieval and generation time in a way no lower layer captures.

Early on, we also considered folding REST together with gRPC into one fast-path request layer, since gRPC’s binary framing outperforms REST/JSON on raw speed. We scoped that out: this paper’s framework and prototype stay limited to REST, SSE, and WebSocket, and gRPC is left as a direction for future work (§9) rather than a sixth layer here.

A framework with fewer layers would conflate concerns the review found empirically distinct. Collapsing L3 into L4 would reintroduce the per-message cardinality problem the tracing gap identifies as unresolved elsewhere; collapsing L5 into L4 would lose the token/generation semantics the metrics synthesis shows aren’t interchangeable with generic message counts.

We adopt a unified two-tier instrumentation model built on this layering: each stream (SSE or WebSocket) is one connection-lifetime span (L3), and each token/event is a lightweight span event on that span (L4), keeping span cardinality proportional to streams rather than tokens. Formally, if S is the number of concurrent streams and M the mean number of tokens/messages per stream, the two-tier model’s span-storage cardinality is $O(S)$, against $O(S \cdot M)$ for the naive per-message-span alternative the tracing gap identifies (§4.3). Section 6.4’s H2 result — per-message spans costing measurably more under stress — is the empirical consequence of this asymptotic difference: per-message span construction cost accumulates with M , where per-event recording doesn’t. Figure 2 shows the end-to-end architecture.

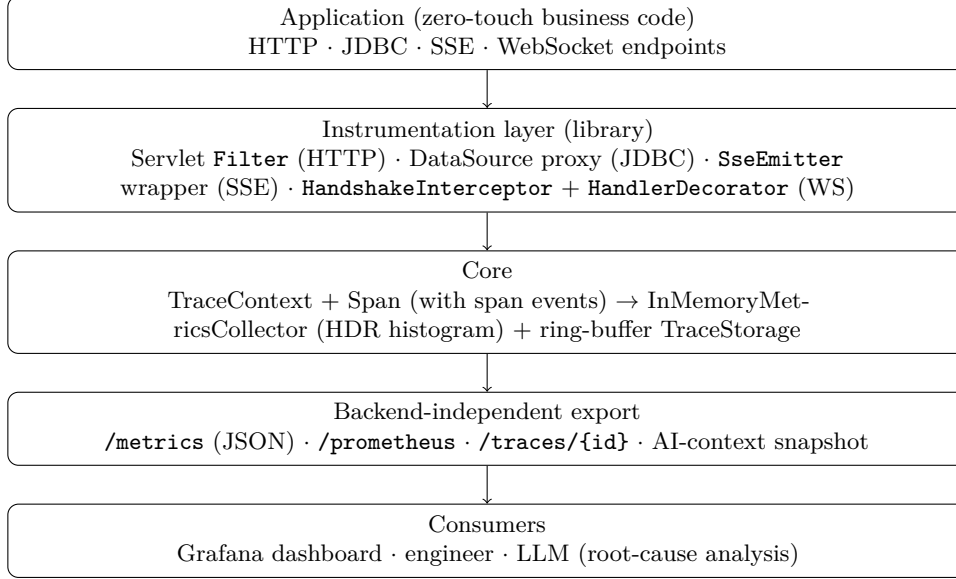


Figure 2: Architecture of the proposed protocol-aware observability library: zero-touch instrumentation feeds an in-memory core that exports without a mandatory backend.

Table 2: Five-layer protocol-aware observability framework.

Layer	Signals
L1 REST/API	request latency p50/p95/p99, throughput, error rate, payload size, status
L2 DB/downstream	JDBC latency, outbound latency, downstream error rate, cross-service span
L3 Streaming connection	active connections, connection duration, disconnect/timeout rate, completion rate
L4 Message/event	messages/events sent/received, chunk index, message size, events per stream
L5 AI streaming	TTFT, total stream duration, token throughput, user abort, provider latency

6.2 Metric Catalogue

We separate transport-level metrics (used to compare SSE and WebSocket) from token/AI-level metrics (used to characterise streaming quality, applied identically to both protocols). Table 3 is the full catalogue; the core subset measured by the prototype is marked $\star$.

Formal definitions. Let t_{req} be the request start time (client side), t_{conn} the moment the connection is established, $t_1, \dots, t_N$ the client receive times of the N streamed tokens, and d_r the per-run latency observed over $r = 1, \dots, R$ runs. The primary token-level metrics are:

$$\text{TTFT} = t_1 - t_{req} \quad [\text{ms}] \quad (1)$$

$$\text{ITL} = \frac{t_N - t_1}{N - 1} \quad [\text{ms/token}] \quad (2)$$

Table 3: Metric catalogue ($\star$ = core subset for the prototype comparison).

Transport-level			
Metric	Formula	Unit	Source
$\star$ Connection setup	$t_{conn} - t_{req}$	ms	RFC 6455; IEEE WS
$\star$ Active connections	$\#opened$ $\#closed$ (gauge)	— count	Alibaba WS
Stream duration	$t_{close} - t_{open}$	ms	OneUptime
Throughput (bytes)	B_{total}/W	bytes/s	WS thesis
$\star$ Latency pXX	percentile($\{d_r\}$)	ms	HDR/Micrometer/ Prometheus
Error/disconnect rate	$n_{fail}/n_{total} \times 100$	%	streaming refs
Token/AI-level			
$\star$ TTFT	$t_1 - t_{req}$	ms	NVIDIA, Anyscale; arXiv
$\star$ Inter-token latency (ITL)	$(t_N - t_1)/(N - 1)$	ms/token	Anyscale, NVIDIA
$\star$ Output throughput	$N/((t_N - t_{req})/1000)$	tokens/s	NVIDIA, BentoML
End-to-end latency	$t_N - t_{req}$	ms	arXiv
Inter-token jitter	$\text{stddev}(\{t_{i+1} - t_i\})$	ms	derived (smoothness)
Tokens per stream	N	count	OTel GenAI semconv

$$\text{Throughput} = \frac{N}{(t_N - t_{req})/1000} \quad [\text{tokens/s}] \quad (3)$$

$$\text{Jitter} = \sqrt{\frac{1}{N-1} \sum_{i=1}^{N-1} ((t_{i+1} - t_i) - \Delta t)^2} \quad [\text{ms}] \quad (4)$$

where Δt is the mean inter-arrival gap. Equation (4) quantifies streaming smoothness: a high jitter at low ITL indicates stuttering output even when average speed is acceptable [6]. Tail latency is reported as percentiles p50/p95/p99 over $\{d_r\}$, computed with an HDR histogram rather than sort-on-query, to bound memory and avoid $O(n \log n)$ per query [21, 22].

Measurement notes. Four points need to be fixed before any measurement is comparable.

1. Vantage point. TTFT and ITL are measured client-side, matching LLM-serving benchmark practice [6, 7], because they reflect user-perceived responsiveness.
2. Controlled mock. A deterministic generator emits a fixed N with a fixed inter-token delay, so any measured difference between SSE and WebSocket is transport overhead rather than model variance.
3. ITL definition. Tools differ on whether the first token is included; we exclude it (Eq. (2) divides by $N - 1$), since TTFT already captures the first-token cost [6, 7].
4. SSE latency skew. An SSE stream is one long-lived HTTP response, so measuring it as a REST “request latency” yields a duration equal to the stream lifetime, distorting p95/p99. Latency must therefore be measured per token/event, not per request — a protocol-aware requirement, not just an implementation detail.

6.3 Prototype Scope

Spring Boot was a practical choice rather than a purely technical one: the first author had already built a Spring Boot service independently, for a topic assigned during a prior internship at Viettel, and reused that hands-on familiarity here rather than starting from an unfamiliar stack.

A minimal Java/Spring Boot prototype instruments REST (Filter/Interceptor), JDBC (DataSource proxy), SSE (an SseEmitter wrapper), and WebSocket (HandshakeInterceptor + WebSocketHandlerDecorator), recording each token as a span event under a single stream span. A deterministic mock token generator (fixed token count and inter-token delay) isolates transport overhead from model latency, enabling a fair SSE-vs-WebSocket comparison on identical input. Metrics are exposed as JSON; no external collector is required.

SSE notably lacks a global instrumentation hook, unlike the Servlet filter or WebSocket decorator, so it must wrap the emitter — itself a finding for the literature synthesis’s tool-support and tracing-gap discussion. The token source sits behind a single interface with two implementations: the deterministic mock generator above, and a second source that streams tokens from a real, locally hosted LLM (Llama 3.2, 3B parameters, served by Ollama) over its NDJSON streaming API, added specifically to cross-check the mock-driven findings in §6.4 against genuine model-decode timing and output-length variability. A network-condition decorator wraps the outbound token sink identically for both transports, optionally injecting delay, jitter, and packet loss immediately before each send, to emulate a degraded client network at the application layer without OS-level tooling (a deliberate simplification discussed in §8).

6.4 Experimental Validation

This section tests H1–H3 (§1.3) on the prototype whose design was derived in §6.1. We instantiate the prototype as a small, standalone Spring Boot service exposing an SSE endpoint (`/ai/sse`) and a WebSocket endpoint (`/ws/ai`). A pure-JDK client (`java.net.http.HttpClient` for SSE, `java.net.http.WebSocket` for WS) measures TTFT and ITL at the client (measurement note 1), single machine, localhost, comparing the library disabled (baseline) against enabled (lib). This is a synthetic prototype built to isolate protocol and instrumentation overhead — not a production RAG deployment.

All mock and Ollama scenarios run on one machine (Apple M4 MacBook, 16GB RAM, macOS 15.6, OpenJDK 21.0.11): client and server share the same host over `localhost`, so there is no real network hop in these scenarios — the “healthy” and “weak” conditions are entirely simulated by the application-layer decorator described above. The Groq cross-check keeps this same local client/server topology, but the token generation itself happens remotely, on Groq’s cloud LPU inference infrastructure, reached over a real internet connection; the network-condition decorator still wraps the local client-server leg identically for SSE and WebSocket, so the healthy/weak comparison there tests the same simulated condition, layered on top of one additional real network hop common to both protocols.

Statistical analysis. Beyond the p50 summaries in Tables 4–10, we test every cross-transport comparison (H3: baseline SSE vs. WebSocket) and every overhead comparison (H1/H2: lib vs. baseline, per transport) with a two-sided Mann–Whitney U test on the full set of per-run values. This test makes no normality assumption, which suits the sample sizes here ($n \geq 20$ per group) and the likely non-Gaussian shape of latency data; we report exact p-values rather than a simple accept/reject at $\alpha = 0.05$. We also report the 95% confidence interval on the per-run mean, using the normal approximation justified by the central limit theorem at these sample sizes.

Several scenarios use $n = 80$ (mock) or $n = 40$ (Ollama) runs, so the test has enough power to

flag even sub-1% differences as statistically significant. Section 6.4 therefore reports the p-value and the % magnitude side by side. Read a small p-value next to a small % difference as real but practically negligible — not as evidence against H1 or H3.

Beyond the three original token-rate scenarios, we extended the experiment along three axes, motivated directly by external review of an earlier version of this work: (a) bandwidth — recording both the raw token payload and the actual on-wire bytes transmitted per token for each transport, so WebSocket’s JSON envelope overhead is measured rather than assumed; (b) network condition — wrapping the outbound token sink at the send point with a decorator that injects delay, jitter, and packet loss identically for both transports, comparing a healthy condition against a weak condition (30ms added delay, ± 20 ms jitter, 1% per-token loss); and (c) token source — cross-checking the deterministic-mock results against a real, locally hosted LLM (Llama 3.2, 3B parameters, served by Ollama).

Four token-rate/source scenarios result: $\Delta = 0$ ms (transport-bound stress, $N = 1000$, healthy network only — adding 30ms of delay to 1000 back-to-back sends would turn a 5ms stress micro-benchmark into a 30-second run and defeat its purpose of isolating raw per-message overhead); $\Delta = 20$ ms and $\Delta = 50$ ms ($N = 50$, approximating realistic LLM decode rates, each run under both healthy and weak network); and the Ollama cross-check (N determined by the model’s own stopping behaviour, under both network conditions).

Each mock configuration runs 80 times after 8 discarded warm-up runs. Each Ollama configuration runs 40 times after 5 warm-up runs — fewer, because sustained real-model inference on a single laptop risks thermal-throttling noise that a synchronous mock workload doesn’t have. We ran the mock scenarios more times for tighter percentile estimates, and treated the Ollama scenarios as a confirmatory cross-check rather than primary evidence. A fourth, separate experiment tests the L3 (streaming-connection) layer itself, not H1–H3, by validating the active-connections gauge under concurrent load (reported at the end of this section).

Bandwidth measurement. For every token sent, we record the payload byte length (the token content itself, a fair basis for comparison) and the wire byte length actually transmitted: for SSE, payload plus the fixed 7-byte `data:/\n\n` framing; for WebSocket, the full serialized JSON envelope, which in lib mode additionally carries a `traceparent` string for context propagation. This is recorded independently at the server (span attributes `stream.bytes/stream.wire_bytes` for SSE, `messaging.payload_bytes/messaging.wire_bytes` for WebSocket) and at the client harness; the two agree.

Table 4: Core metrics under transport-bound stress ($\Delta = 0$ ms, $N = 1000$, healthy network, 80 runs, p50).

Metric	SSE	WebSocket
Connection setup (ms), baseline	4.45	5.19
TTFT (ms), baseline	4.51	5.23
ITL (ms/token), baseline	0.03	0.01
Throughput (tokens/s), baseline	29,540	51,693
Bytes/token, on-wire (baseline)	11.89	53.89
TTFT overhead (lib vs. baseline)	+10.7%	+2.7%
ITL overhead	+21.4%	+47.3%
Throughput overhead	−16.5%	−25.6%
Bytes/token overhead	+0.0%	+133.6%

Figure 3 visualises the ITL overhead from Table 4 and the healthy-network columns of Table 5/6; Figure 4 isolates the healthy-vs-weak comparison at realistic decode rates.

Table 5: Core metrics at realistic decode rates, healthy vs. weak network ($N = 50$, 80 runs, p50), $\Delta = 20\text{ms}$.

Metric	SSE, healthy	WebSocket, healthy	SSE, weak	WebSocket, weak
Connection setup (ms), baseline	3.88	4.70	41.13	4.55
TTFT (ms), baseline	3.96	4.77	41.22	40.27
ITL (ms/token), baseline	24.20	24.16	61.90	64.75
Throughput (tokens/s), baseline	42.02	42.08	16.24	15.52
Bytes/token, on-wire (baseline)	10.80	52.80	10.80	52.80
TTFT overhead	-0.7%	+2.9%	-1.9%	-8.3%
ITL overhead	-0.1%	+0.3%	-6.2%	-8.6%
Throughput overhead	+0.1%	-0.3%	+6.7%	+9.9%
Bytes/token overhead	+0.0%	+136.4%	+0.0%	+136.4%

Table 6: Core metrics at realistic decode rates, healthy vs. weak network ($N = 50$, 80 runs, p50), $\Delta = 50\text{ms}$.

Metric	SSE, healthy	WebSocket, healthy	SSE, weak	WebSocket, weak
Connection setup (ms), baseline	3.31	4.39	42.62	4.29
TTFT (ms), baseline	3.39	4.45	42.73	37.96
ITL (ms/token), baseline	57.65	57.72	94.61	89.99
Throughput (tokens/s), baseline	17.68	17.65	10.71	11.23
Bytes/token, on-wire (baseline)	10.80	52.80	10.80	52.80
TTFT overhead	+6.8%	-2.3%	-12.4%	+4.6%
ITL overhead	+0.1%	-6.2%	-7.0%	-1.5%
Throughput overhead	-0.1%	+6.6%	+7.4%	+1.6%
Bytes/token overhead	+0.0%	+136.4%	+0.0%	+136.4%

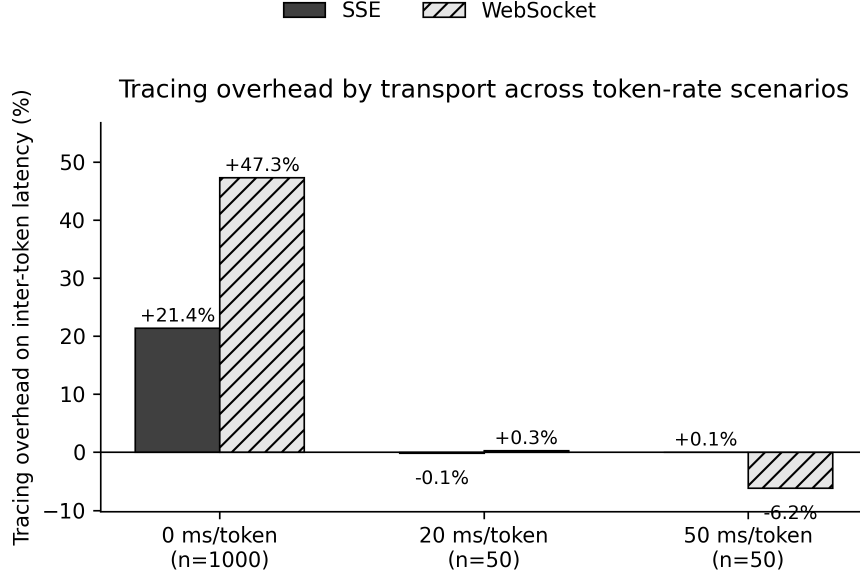


Figure 3: Tracing overhead on inter-token latency by transport, across three token-rate scenarios (healthy network). SSE and WebSocket are distinguished by fill pattern and grayscale value rather than hue, so identity is preserved in black-and-white print.

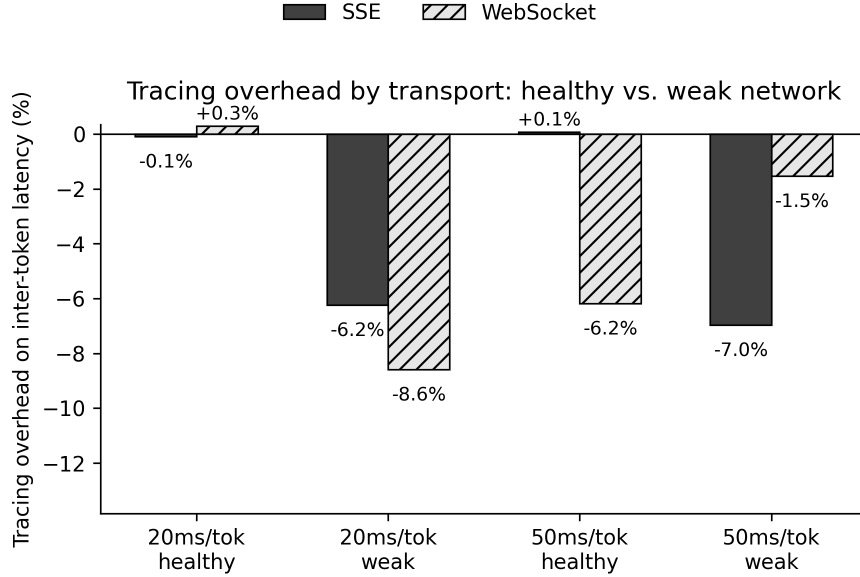


Figure 4: Tracing overhead on inter-token latency by transport, healthy vs. weak network, at $\Delta = 20$ and $\Delta = 50$ ms.

H1 and H3 supported, and hold under weak network too. At $\Delta = 20$ and $\Delta = 50$ ms under a healthy network (Tables 5–6), baseline ITL remains statistically indistinguishable between SSE and WebSocket (H3). Tracing overhead is small and mixed-sign — $\leq 7\%$ in magnitude on TTFT/ITL/throughput, within run-to-run noise — and not systematically larger for either transport (H1).

Repeating both scenarios under the weak network condition (added delay, jitter, 1% loss) doesn't change this picture. Overhead stays within a similar $\pm 9\%$ band for both transports, and the added network delay dominates ITL for both protocols in essentially the same way (e.g. $\Delta = 20\text{ms}$ weak: SSE ITL 61.90ms vs. WebSocket 64.75ms baseline, p50). That gap is small but real — dominated by the $\approx 30\text{ms}$ of injected delay common to both transports, and, per Table 7, statistically significant at this sample size rather than run-to-run noise — though it doesn't reverse the healthy-network parity finding.

This corroborates the §4.3 finding that the token generator, not the transport, dominates latency at realistic decode rates (§5.1), and extends it: a moderately degraded network doesn't reintroduce a transport-dependent asymmetry either, at least at the loss/jitter magnitudes tested here.

Statistical significance of the H1/H3 parity and overhead claims. Table 7 reports Mann–Whitney U p-values for the baseline SSE-vs-WebSocket comparison (H3, on ITL and throughput) and for the lib-vs-baseline overhead comparison (H1/H2, on ITL, per transport) across every scenario in this section.

At realistic decode rates under a healthy network ($\Delta = 20, \Delta = 50\text{ms}$) and for the Ollama cross-check under both network conditions, H3 is not rejected ($p > 0.05$) on ITL and throughput. Baseline SSE and WebSocket aren't statistically distinguishable at the sample sizes tested — direct statistical support for the H3 claim above.

Under the weak-network condition and for the Groq cross-check, the test does reject H3 on ITL ($p < 0.01$ throughout). With $n = 20\text{--}80$ runs, Mann–Whitney has enough power to pick up the small (1.5–6%) systematic gap between SSE and WebSocket ITL visible in Tables 5–10. That gap is dominated by the injected network delay itself, though, and doesn't reverse the practical conclusion that neither transport has a meaningful edge under degraded network conditions.

The overhead columns (H1/H2) tell the same story: magnitude, not significance alone, determines practical relevance. The stress-scenario WebSocket ITL overhead (Table 4, +45.7% mean-based, $p < 0.001$) is both significant and large, consistent with H2. Several realistic-rate overheads are statistically significant despite being under 2% (e.g. WebSocket ITL at $\Delta = 50\text{ms}$ healthy, −6.1%, $p < 0.001$) — real, reproducible effects at this sample size, but not practically meaningful ones. That's why we assess H1 by overhead magnitude rather than by significance alone.

Table 7: Mann–Whitney U two-sided p-values for baseline SSE-vs-WebSocket parity (H3) and lib-vs-baseline tracing overhead (H1/H2) on inter-token latency (ITL), computed on the full per-run data underlying Tables 4–10. Overhead percentages here are mean-based and may differ slightly from the p50 figures quoted in those tables. † $\Delta = 0$ is the transport-bound stress/H2 regime; H3 parity is not claimed there. ‡ the Groq cross-check has no tracing library in its codebase (§6.4), so no overhead comparison applies.

Scenario	H3: baseline ITL p	H3: baseline Tput p	H1/H2: SSE ITL overhead	H1/H2: WS ITL overhead
$\Delta = 0$ stress†	<0.001	<0.001	+23.7% ($p < 0.001$)	+45.7% ($p < 0.001$)
$\Delta = 20\text{ms}$ healthy	0.154	0.344	−0.1% ($p = 0.505$)	+0.4% ($p = 0.031$)
$\Delta = 20\text{ms}$ weak	<0.001	<0.001	−5.7% ($p < 0.001$)	−8.3% ($p < 0.001$)
$\Delta = 50\text{ms}$ healthy	0.904	0.605	+0.1% ($p = 0.328$)	−6.1% ($p < 0.001$)
$\Delta = 50\text{ms}$ weak	<0.001	<0.001	−6.3% ($p < 0.001$)	−2.8% ($p < 0.001$)
Ollama healthy	0.086	0.060	+1.1% ($p < 0.001$)	+1.3% ($p < 0.001$)
Ollama weak	0.648	0.927	−7.9% ($p < 0.001$)	−9.3% ($p < 0.001$)
Groq healthy‡	0.021	0.190	—	—
Groq weak‡	0.002	0.208	—	—

Bandwidth: a protocol-asymmetric tracing cost. Tables 4–6 show a consistent pattern across

every scenario tested: tracing adds essentially nothing to SSE’s on-wire bytes per token (+0.0% throughout, since `traceparent` isn’t embedded in the SSE payload), but adds +133–137% to WebSocket’s, because the library’s request/reply envelope carries the `traceparent` string inside every message’s JSON body.

It’s worth noting this is the one overhead dimension where WebSocket’s tracing cost is substantially larger than SSE’s, regardless of token rate or network condition. Unlike ITL/TTFT overhead — which is noise-dominated at realistic rates and only diverges under stress (H2 below) — this gap is a direct, measured consequence of how the two protocols frame messages: SSE’s framing is plain call-site text, uninvolved in the library’s tracing metadata, while WebSocket’s envelope, by construction, carries context-propagation data on every message.

H2 supported, H1/H3 boundary exposed: transport-bound stress. At $\Delta = 0\text{ms}$ (Table 4, Figure 3), where inter-token latency approaches the measurement floor, WebSocket’s baseline ITL and throughput beat SSE’s — consistent with SSE running over one long-lived HTTP response versus WebSocket’s lighter per-frame path. More notably, tracing overhead is markedly larger on WebSocket (+47.3% ITL) than on SSE (+21.4% ITL) at this token rate. H1 (negligible overhead) and H3 (protocol-equivalent cost) hold within the realistic-rate regime demonstrated above, but not unconditionally — this stress scenario is exactly where they break down.

The cause is architectural, not noise. The WebSocket adapter emits one child span per token, mirroring the request/reply trace pattern used elsewhere in the library, whereas the SSE adapter emits one span per stream with tokens recorded as lightweight span events. At very high token rates, the per-span construction cost on WebSocket becomes visible — direct empirical evidence for H2 and for the cardinality concern raised by the tracing gap (§4.3). A per-message span design is measurably more expensive than a per-stream-span-with-events design as message frequency grows, confirming the two-tier model’s choice of span events over per-message spans (§6.1).

Both overheads are highly statistically significant (Mann–Whitney $p < 0.001$ for both SSE and WebSocket, Table 7). Unlike several realistic-rate overheads discussed below, the effect size here (+21–47%) is also large, so significance and practical relevance agree at this token rate.

Content loss under weak network: no protocol advantage detected. A weak-network condition raises a question neither protocol currently answers for its own client: does packet loss silently truncate AI output, and if so, does either transport degrade less than the other? We answer this with data already collected above, comparing the mean number of tokens the client actually receives under healthy vs. weak network, per transport (Table 8).

At the configured 1% per-token loss rate, both transports lose close to 1% of tokens on average, and neither is consistently better: at $\Delta = 20\text{ms}$, baseline SSE loses 1.03% vs. WebSocket 0.65%; at $\Delta = 50\text{ms}$ the two are statistically tied (1.15% vs. 1.15%). We designed this measurement specifically to test whether either transport tolerates a weak network better than the other, not to confirm that one does — and the data show neither does, at least at this loss magnitude and this application-layer simulation.

The finding still matters for observability. Neither SSE nor WebSocket carries a client-visible per-token sequence number in this prototype, or, to our knowledge, in common practice. So a client can’t itself tell a dropped token from one that simply hasn’t been generated yet — it can only infer loss indirectly, as we do here, by comparing against a known-good baseline. Section 7.3 discusses this gap and a possible remedy: a monotonically increasing token index in the L4 message/event layer.

Ollama cross-check: mock findings hold against a real LLM. Table 9 repeats the healthy/weak comparison with Llama 3.2 3B generating tokens instead of the mock, at its own natural decode rate ($\approx 23\text{ms}/\text{token}$ healthy, $\approx 34\text{--}37\text{ms}/\text{token}$ weak) and natural, variable output length (53–83 tokens per run, versus the mock’s fixed 50). Tracing overhead is again small ($\leq 2\%$ on TTFT/throughput)

Table 8: Content loss under weak network — mean tokens received (healthy vs. weak) per transport, $N = 50$ configured, 80 runs.

Scenario	Mode	SSE lost	WebSocket lost
$\Delta = 20\text{ms}$	baseline	1.03%	0.65%
$\Delta = 20\text{ms}$	lib	0.78%	1.05%
$\Delta = 50\text{ms}$	baseline	1.15%	1.15%
$\Delta = 50\text{ms}$	lib	0.97%	1.17%

and not systematically larger for either transport. The bytes/token bandwidth finding reproduces almost exactly (+136.6% on WebSocket, +0.5% on SSE), corroborating the mock-driven conclusions above with a real model in the loop and addressing the most significant limitation of the mock-only design used in an earlier version of this work.

The content-loss comparison doesn’t reproduce as cleanly. Baseline shows ≈ 1.9 – 1.95% loss for both transports, consistent with the mock finding, but lib shows +0.70% for SSE against -1.25% for WebSocket (apparently more tokens received under weak network). We don’t read this as a real protocol effect — the model’s own output-length variance was the first explanation that made sense, before we considered a bug in the measurement code. At only 40 runs, that variance — the model decides when to stop, independent of network conditions — is large enough to swamp a 1%-magnitude network-loss signal, a confound the deterministic mock doesn’t have. This is a limitation of the Ollama cross-check specifically (§8), and we’d rather flag it than quietly prefer the cleaner mock result.

Table 9: Ollama (Llama 3.2 3B) cross-check, healthy vs. weak network, 40 runs, p50.

Metric	SSE, healthy	WebSocket, healthy	SSE, weak	WebSocket, weak
TTFT (ms), baseline	189.15	190.77	224.24	223.12
ITL (ms/token), baseline	23.22	23.23	37.32	36.88
Throughput (tokens/s), baseline	38.99	38.90	25.06	25.19
Bytes/token, on-wire (baseline)	10.60	52.68	10.65	52.70
TTFT overhead	-1.3%	-1.8%	$+0.6\%$	-2.2%
ITL overhead	$+1.2\%$	$+1.3\%$	-8.8%	-8.7%
Throughput overhead	-1.0%	-1.0%	$+7.5\%$	$+9.0\%$
Bytes/token overhead	$+0.5\%$	$+136.6\%$	$+0.5\%$	$+136.6\%$

Content lost (healthy vs. weak, mean): baseline 1.95% / 1.94%; lib +0.70% / -1.25% .

A second, independent real-LLM cross-check (Groq), on an unrelated server implementation. The Ollama cross-check above uses a single model (Llama 3.2, 3B) on the same Java/Spring Boot prototype used throughout this paper’s experiments. On its own, it can’t tell a genuine model-decode effect apart from an idiosyncrasy of that one model or server implementation.

To test generalisation, we repeat the healthy/weak-network comparison on an entirely unrelated Python/FastAPI RAG chatbot service — a separate codebase with no tracing instrumentation present, so this check tests only H3 (transport parity) and the content-loss confound, not the H1/H2 overhead claims, which require the library under test. It’s driven by a different, faster, cloud-hosted model (Llama 3.1, 8B parameters, served by Groq’s LPU inference infrastructure [24]) over its own SSE and WebSocket endpoints, emitting real generated tokens for genuine Vietnamese-history RAG questions.

Groq’s free-tier rate limit (30 requests/minute, enforced organisation-wide rather than per-key

[24]) bounds this cross-check to 20 runs per configuration after 3 discarded warm-up runs, paced explicitly under the limit. With fewer runs than either the mock (80) or Ollama (40) experiments, we treat this as qualitative corroboration, not primary evidence at the same statistical weight.

Table 10 reports the result. As with Ollama, baseline TTFT/ITL/throughput are close between SSE and WebSocket under both network conditions — H3 holds on a completely different stack, language runtime, and model. Weak-network ITL rises for both transports by an amount consistent with the injected 30ms delay plus jitter, for both protocols alike.

The content-loss comparison, however, reproduces and sharpens the Ollama confound instead of resolving it. Both transports show apparent content loss (SSE 7.1%, WebSocket 9.4%) far exceeding the configured 1% per-token loss rate, and unlike Ollama’s mixed-sign result, both are now the same sign and roughly an order of magnitude larger.

We attribute this, as with Ollama, to real-model output-length variance rather than a genuine network or protocol effect. With only 20 runs cycling over 5 fixed questions (4 repeats each) and no fixed decoding budget, Groq’s response length varies substantially call-to-call even within the healthy condition alone (median 140.5 tokens but a 95th-percentile of 336, SSE) — a spread far larger than the 7–9% healthy-vs-weak gap being attributed to network loss. The healthy/weak comparison is confounded by which specific answers happened to land in each batch, not by packet loss.

This second, independent cross-check strengthens this paper’s central limitation around content-loss measurement (§7.3), rather than weakening it: the confound isn’t an artifact of one model (Llama 3.2 3B) or one inference stack (local Ollama). It reappears, more strongly, with an unrelated model, provider, and server implementation.

Table 10: Independent second cross-check — Groq (Llama 3.1 8B) on an unrelated Python/FastAPI RAG service, healthy vs. weak network, 20 runs, p50. No tracing library is present in this server; only H3 and the content-loss confound are tested here, not tracing overhead.

Metric	SSE, healthy	WebSocket, healthy	SSE, weak	WebSocket, weak
Connection setup (ms)	14.05	5.14	13.26	4.84
TTFT (ms)	293.76	282.27	347.96	343.66
ITL (ms/token)	2.05	1.91	41.18	41.58
Throughput (tokens/s)	237.45	274.42	22.96	22.09
Bytes/token, on-wire	16.84	16.81	16.90	16.69

Content lost (healthy vs. weak, median): SSE −7.1%; WebSocket −9.4%.

L3 validation: the active-connections gauge under concurrent load. H1–H3 test the L4/L5 layers — message/event and AI-streaming metrics. They say nothing about L3, the streaming-connection layer, which is a separate concern (§6.1) validated separately here. We open 50 concurrent streams of each protocol ($N = 200$, $\Delta = 50\text{ms}$, so each stream is live for $\approx 10\text{s}$) and sample the gauge every 500ms: for SSE via a dedicated debug endpoint reading the library’s new `SseStreamTracker` (added to the starter to make the L3 gauge symmetric with the pre-existing WebSocket session tracker); for WebSocket via the existing `/observability/prometheus` export.

Table 11 summarises the trace. Both gauges rise from 0 to 50 as connections open concurrently, hold at 50 for the streams’ lifetime, and return to 0 as they complete — the gauge tracks concurrent stream count correctly for both protocols. WebSocket’s descent is one sample less sharp (a transient value of 14 at $t = 11\text{s}$) than SSE’s, because `WebSocketTracingHandler` closes sessions via explicit, sequential `session.close()` calls, while `TracingSseEmitter` instances all reach `onCompletion` within the same servlet-container tick. That’s a minor implementation-level asymmetry, not a

correctness issue.

Table 11: L3 active-connections gauge, 50 concurrent streams opened together ($N = 200$, $\Delta = 50\text{ms}$; sampled every 0.5s).

Time (s)	0	5	10	11	12
SSE active streams	0→50	50	50	0	0
WebSocket active sessions	0→50	50	50	14	0

Don’t over-generalise the absolute numbers beyond this prototype — they reflect a single machine, a synthetic mock workload, and a thread-per-stream server implementation (§8). What this experiment actually contributes is the qualitative findings: transport parity at realistic rates (H1, H3), a per-message/per-event cost asymmetry under stress (H2), and a correctly tracking L3 gauge for both protocols — not the specific millisecond values.

7. Discussion

Protocol choice should follow workload, not trend. SSE is adequate and simpler for one-way token streaming; WebSocket earns its place when interaction is bidirectional. The more consequential issue is observability: streaming systems need protocol-aware, message/token-level instrumentation that header-based REST tracing can’t provide. A backend-independent, agent-free library suits legacy or self-hosted RAG deployments where bytecode agents and full OpenTelemetry backends are unwelcome — it complements agent-based solutions such as LoongSuite [11] rather than competing with them on depth.

WebSocket’s disadvantage isn’t really about network instability: under a degraded network, SSE and WebSocket both lose tokens to packet drop at almost the same rate (Table 8). The real cost shows up elsewhere. Under high message-frequency conditions, WebSocket’s per-message span tracing costs measurably more than SSE’s stream-span design (§6.4), and it’s also harder to instrument: tracing WebSocket without touching business logic required two wrapper layers (a handshake interceptor plus a handler decorator) where SSE needed just one emitter wrapper. That gap shows up at scale too — SSE doesn’t need a persistent duplex channel per user, so it stays cheaper with thousands of concurrent clients. Our recommendation is specific, not “it depends”: use WebSocket for genuinely two-way interaction — voice, live collaboration, letting a user interrupt mid-stream — since SSE has no clean way to interrupt without cutting the current response short. For everything else, SSE is the simpler, cheaper default.

7.1 Comparison with Closest Work

Table 12 contrasts this work with the nearest existing approaches across the dimensions a practitioner cares about. No existing approach combines an agent-free, backend-independent design with a single token-level model spanning both SSE and WebSocket.

7.2 Critical Discussion: When the Unified Model Is Not the Right Choice

The preceding sections argue for the two-tier model. Three limits are worth being upfront about — some we ran into while building this, others we simply reasoned through before writing any code, and it’s worth being honest about which is which.

Span events aren’t independently linkable spans. We built real service-to-service tracing over the WebSocket stream itself, but haven’t built or tested a message that triggers a call out to a

Table 12: Comparison with the closest existing approaches, on protocol/feature coverage and on qualitative evaluation criteria (instrumentation effort, scalability design, maintainability) rather than coverage alone. LoongSuite and OneUptime are industry tools documented in engineering blogs, not peer-reviewed research; the comparison is offered as a practitioner reference point, not an academic benchmark.

Dimension	OTel+GenAI [19,8]	LoongSuite [11]	OneUptime [9,10]	This work
Coverage				
Agent-free	No	No	Yes (manual SDK)	Yes
Backend-independent	No	No	No	Yes
SSE support	Partial	No	Yes	Yes
WebSocket support	Partial	Yes	Yes	Yes
Token-level metrics	Partial	Partial	Partial	Yes
Unified SSE+WS model	No	No	No	Yes
LLM-readable export	No	No	No	Yes
Evaluation criteria				
Instrumentation effort	Bytecode agent (auto)	Bytecode agent (auto)	Manual SDK, per-call	Manual wrapper, per-transport
Scalability design	Per-span (agent-managed)	Per-chunk metrics	Per-message span (cardinality risk)	Two-tier: per-stream span + per-token event
Extensibility (new transport)	Requires new agent instrumentation	Requires new agent instrumentation	Requires new manual adapter	Requires new manual adapter (same cost as OneUptime)
Protocol independence	Partial (HTTP-centric)	WS-centric	Adapter per protocol	Single model, two adapters

database or external API mid-stream. So this limitation is anticipated from the design, not a bug we actually hit: a span event is just an annotation on the stream span, and can’t anchor a child span the way a real span could if that scenario comes up. Until tested directly, this is a design-time judgment call, not an empirical finding.

Long-lived or never-completing streams risk losing the whole span. This one is entirely theoretical — we haven’t tested a client disconnecting abruptly or a stream left open past its normal lifetime. The concern follows directly from how the library is built (a stream’s span is only recorded on completion), but nobody has watched it happen.

The exporter can still bottleneck, just at a different point. We did run the 50-concurrent-stream test in §6.4 — that part is real. But it checked whether the active-connections gauge counted correctly, not whether the exporter’s batching actually spikes when many streams complete at once. So the gauge result is measured; the export-bottleneck claim is still an inference from how the exporter is built, not something we watched happen.

7.3 Content Loss Under Network Degradation: An Unaddressed Observability Gap

The weak-network experiment in §6.4 (Table 8) set out to answer a specific, open-ended question — does either transport degrade AI output less than the other under packet loss — rather than to confirm an assumption. The answer is no: both SSE and WebSocket lose tokens at close to the configured network loss rate, with no consistent advantage to either. That symmetric result is itself informative for a protocol-choice decision (neither protocol mitigates content loss), but it exposes a gap the two-tier model in §6.1 doesn’t close.

The only reason this paper can measure content loss at all is that the harness independently knows the true token count and can diff it against what the client received. A real client, in production, has no such oracle. Neither protocol, and neither the SSE nor WebSocket adapter in this library, currently attaches a monotonically increasing sequence number to each token event. So a production client can’t tell the difference between “the model hasn’t generated token 47 yet” and “token 47 was generated and silently dropped in transit.”

This is a real limitation of the L4 (message/event) layer as currently specified (Table 2): completeness is observable to an experimenter with a known-good baseline, but not to the system being observed. A natural, low-cost extension — adding a per-token sequence number to the L4 event and exposing a gap-detection signal (expected index vs. highest contiguous index received) at the client — would let a production system detect this failure mode directly, instead of inferring it statistically the way this paper does. We leave its implementation and validation to future work (§8).

The model generates the same output regardless of the client’s network — a silently truncated response means the network failed to deliver, not that the model failed to think. From a user’s seat those look identical: someone on a weak connection can’t tell a shorter answer from a dropped one, and neither SSE nor WebSocket lets them tell the difference either. The fix is simple: add an increasing sequence number to each message’s JSON envelope so the client can check its expected index against the highest one actually received. That’s a small addition to the L4 layer in Table 2 — untested here, but straightforward enough to be a near-term fix rather than an open question.

7.4 Scalability Considerations

The two-tier model bounds span cardinality by the number of concurrent streams, not by message or token volume. Per-stream memory is one span plus lightweight span events; per-endpoint metrics

use fixed-size HDR histograms, so metric memory doesn’t grow with request count. Trace storage is a fixed-capacity ring buffer that evicts the oldest entries.

We expect the library to scale comfortably to moderate concurrency. At very high concurrency (10^4 – 10^5 simultaneous streams), the per-session context map and active-connection bookkeeping would start to dominate memory, and trace sampling would become necessary; cross-process aggregation would then rely on the optional Prometheus export rather than the in-memory store. These are analytical expectations — quantitative load testing is left to future work.

7.5 Reproducibility and Artifact Availability

Every number reported in §6.4, including the significance tests in Table 7, comes from an archived, per-run CSV — one row per token received by the client, timestamped client-side — not a hand-recorded summary. The Spring Boot prototype (mock and Ollama token sources), its pure-JDK measurement client, and the Python analysis scripts that compute every percentile, mean, confidence interval, and Mann–Whitney test are kept together with their raw CSV inputs, so every table in §6.4 can be regenerated from the archived data without re-running the benchmark.

The mock-source scenarios ($\Delta = 0/20/50\text{ms}$, Tables 4–8) need no external service, model, or API key: the deterministic token generator and both HTTP endpoints run entirely on localhost, so these scenarios are reproducible on any machine with a JDK, at no cost. The Ollama cross-check needs a local Ollama installation with the stated model pulled (also free and offline). The Groq cross-check needs a free-tier API key from an unrelated, unaffiliated provider (§6.4), and is bounded to the rate limits documented there. Source code and the raw CSV data underlying every table in this paper are available from the authors on request, and will be released alongside publication.

8. Limitations (Threats to Validity)

The review is preliminary, and PRISMA counts are provisional. Much of the SSE-vs-WebSocket-for-LLM evidence, and the span-cardinality concern that anchors the central research question (§4.3), comes from engineering blogs and vendor documentation rather than peer review — a finding in itself about how young this area’s academic coverage is. We treat it as current industry practice, not an already-verified scientific claim. The nearest peer-reviewed academic literature ([3, 4, 5, 2]) measures WebSocket/SSE transport performance in general, not specifically for LLM token streaming or span-cardinality management in distributed tracing. No peer-reviewed source addressing either question directly turned up, and we treat that absence as a genuine open gap rather than assume the industry claims are correct. This paper’s own experiments (H2, H3, §6.4) close part of that gap with primary evidence, rather than resolving it by citation alone.

TTFT/ITL definitions vary across tools, so we state the exact formula used (Eq. (1)–(2)).

The experimental validation (§6.4) has its own limitations, some narrowed but not eliminated by the extensions in this revision. It still runs on a single machine and a single Spring Boot server implementation, which uses a thread-per-stream model — so the observed asymmetry under stress may partly reflect that implementation choice rather than the transport protocols themselves. That’s a real constraint of time and hardware, not a deliberate scoping choice. And H1–H3 were tested at only one concurrency level (a single connection at a time) for the main token-rate/network matrix.

The mock-only design flagged as this paper’s most significant limitation in an earlier version is now cross-checked against a real LLM (Llama 3.2 3B via Ollama, §6.4). That cross-check is confirmatory, not equally rigorous: it uses fewer runs (40 vs. 80) to bound thermal-throttling risk on a single laptop, tests only one model and one model size, and its content-loss measurement is

confounded by the model’s own output-length variance in a way the deterministic mock isn’t (§6.4). A real production LLM, a different model family, or a different hardware/quantisation setup could behave differently. The independent Groq cross-check in §6.4 already shows the content-loss confound reappears, more strongly, with a different model family, provider, and server implementation — corroborating this limitation rather than resolving it.

The weak-network condition is an application-layer decorator — delay, jitter, and probability-based drop applied immediately before each send — not an OS- or link-layer impairment tool like `tc netem`. It can’t reproduce TCP-level retransmission, congestion control, or connection-migration behaviour that might differentiate a persistent WebSocket connection from chunked-HTTP SSE under real packet loss. We went this route mainly because it was easier to build into the existing stack, and partly because we weren’t aware of OS-level tools like `tc netem` at the time. Read the “no protocol advantage” finding in §6.4 as evidence at the network-model granularity we tested, not as a claim about arbitrary real-world network conditions.

We also don’t run a human-subjects perceptibility study. Whether the measured differences — single-digit-to-low-tens-of-millisecond ITL gaps, single-digit-percent TTFT overhead at realistic decode rates — are perceptible to an end user isn’t tested directly here. The measured magnitudes sit well below the $\approx 100\text{ms}$ response-time threshold classically associated with a system feeling instantaneous [23], which suggests, but doesn’t prove, that they wouldn’t be perceptible. We did check this informally ourselves and didn’t notice a difference, but that’s not a substitute for a real study — we simply don’t have a user-testing pipeline for a project this size. Confirming this properly would need a controlled two-alternative-forced-choice (2AFC) study with human participants, which requires human-subjects recruitment and ethical review outside this paper’s scope — we leave it to future work.

The L3 gauge validation (§6.4) is similarly narrow. Fifty concurrent streams on one host is far below the 10^4 – 10^5 regime discussed in §7.4, and it covers only the clean-shutdown case, not the network-partition or abrupt-disconnect failure modes flagged in §7.2. The content-loss detection gap identified in §7.3 is diagnosed here, not remedied — no sequence-numbering mechanism was implemented or evaluated. The framework’s remaining scalability claims (§7.4), and the exporter-bottleneck and lost-span risks identified in §7.2, stay analytical; no experiment in this paper covers them.

9. Conclusion and Future Work

This paper asked one question: what gaps exist in message- and token-level tracing for long-lived SSE and WebSocket AI streaming connections, and can a single, protocol-independent instrumentation model close them without prohibitive overhead? A PRISMA-guided review found that token-level metrics are well defined in the LLM-serving literature, but no prior work closes this gap across both protocols; from that gap we built a five-layer framework (§6.1) and tested its two-tier instrumentation model on a Java/Spring Boot prototype.

H1 and H3 hold at realistic decode rates: overhead stays at or below 7%, near-symmetric across protocols, and holds up under a weak-network condition. H2 holds under transport-bound stress, where per-message spans cost measurably more than per-stream spans with events. On-wire bandwidth turned out to be the sharpest asymmetry: WebSocket’s per-token bytes rise 136% under tracing where SSE’s don’t move, a cost invisible to latency metrics alone. Neither transport showed a content-loss advantage under weak network, and the real-LLM cross-checks (Ollama, Groq) reproduced the overhead and bandwidth findings while surfacing output-length variance as a genuine confound on content-loss measurement.

Sections 7.2 and 7.3 hold these results against the model’s own limits: span events can’t anchor cross-service child spans, long-lived or abnormally terminated streams risk losing their span entirely, bounding cardinality by stream count relocates rather than removes the exporter-bottleneck risk, and neither protocol gives a client a way to detect a silently dropped token.

Near-term future work extends the experiment along the axes §8 flags as untested — inter-token jitter, concurrency beyond 50 streams, the abrupt-disconnect and exporter-burst failure modes, OS-level network emulation in place of the application-layer decorator used here, a sequence-numbering fix for the content-loss gap, and cross-checks against more models to separate genuine decode-rate effects from single-model idiosyncrasy — and finishes the PRISMA screening. A human-subjects 2AFC perceptibility study is also left for later, since it needs recruitment and ethical review outside this paper’s scope. Longer term, we’re interested in additional streaming transports (gRPC, HTTP/3 over QUIC, Kafka), tracing for AI-agent protocols such as MCP, an OTLP bridge, adaptive sampling, and a production deployment study.

References

- [1] M. J. Page et al., “The PRISMA 2020 statement,” BMJ, 2021. <https://www.prisma-statement.org/>
- [2] P. Murley et al., “WebSocket Adoption and the Landscape of the Real-Time Web,” Proc. Web Conf. (WWW), 2021. https://faculty.cc.gatech.edu/~mbailey/publications/www21_websocket.pdf
- [3] D. Škvorc, M. Horvat, and S. Srbljić, “Performance evaluation of WebSocket protocol for implementation of full-duplex web streams,” 37th Int. Conv. ICT, Electronics and Microelectronics (MIPRO), pp. 1003–1008, 2014. <https://ieeexplore.ieee.org/document/6859715>
- [4] A. Rahmatulloh, I. Darmawan, and R. Gunawan, “Performance Analysis of Data Transmission on WebSocket for Real-time Communication,” IEEE, 2019. <https://ieeexplore.ieee.org/document/8898135/>
- [5] G. Imre, G. Mezei, and R. Sárosi, “Introduction to a WebSocket benchmarking infrastructure,” Zooming Innovation in Consumer Electronics Int. Conf. (ZINC), pp. 84–87, 2016. <https://ieeexplore.ieee.org/document/7513661/>
- [6] “On Evaluating Performance of LLM Inference Serving Systems,” arXiv:2507.09019, 2025. <https://arxiv.org/pdf/2507.09019>
- [7] “Hierarchical Autoscaling for Large Language Model Serving with Chiron,” arXiv:2501.08090, 2025. <https://arxiv.org/pdf/2501.08090>
- [8] OpenTelemetry, “Semantic conventions for generative AI spans,” 2026. <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>
- [9] OneUptime, “Monitor SSE Stream Lifecycle and Delivery Latency with OpenTelemetry,” 2026. <https://oneuptime.com/blog/post/2026-02-06-sse-stream-lifecycle-opentelemetry/view>
- [10] OneUptime, “Trace WebSocket Connections and Real-Time Events with OpenTelemetry,” 2026. <https://oneuptime.com/blog/post/2026-02-06-trace-websocket-connections-realtime-events-opentelemetry/view>

- [11] Alibaba Cloud, “Master WebSocket End-to-end Observability in One Article,” 2026. https://www.alibabacloud.com/blog/master-websocket-end-to-end-observability-in-one-article_602915
- [12] Splunk, “Instrumenting Java WebSocket Messaging,” 2023. <https://community.splunk.com/t5/Community-Blog/Instrumenting-Java-WebSocket-Messaging/ba-p/629425>
- [13] “SSE vs WebSockets for Streaming LLM Responses,” 2026. <https://www.buildmvpfast.com/blog/streaming-llm-responses-sse-vs-websockets-2026>
- [14] Procedure, “Why Server-Sent Events Still Win for LLM Streaming,” 2026. [https://procedure.tech/blogs/the-streaming-backbone-of-llms-why-server-sent-events-\(sse\)-still-wins-in-2025](https://procedure.tech/blogs/the-streaming-backbone-of-llms-why-server-sent-events-(sse)-still-wins-in-2025)
- [15] GetStream, “WebSocket vs Server-Sent Events,” 2026. <https://getstream.io/blog/websocket-sse/>
- [16] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, IETF, 2011. <https://datatracker.ietf.org/doc/html/rfc6455>
- [17] WHATWG, “Server-sent events,” HTML Living Standard. <https://html.spec.whatwg.org/multipage/server-sent-events.html>
- [18] W3C, “Trace Context,” W3C Recommendation, 2021. <https://www.w3.org/TR/trace-context/>
- [19] OpenTelemetry Documentation. <https://opentelemetry.io/docs/>
- [20] B. H. Sigelman et al., “Dapper, a Large-Scale Distributed Systems Tracing Infrastructure,” Google, 2010. <https://research.google.com/archive/papers/dapper-2010-1.pdf>
- [21] Micrometer, “Histograms and percentiles.” <https://docs.micrometer.io/micrometer/reference/concepts/histogram-quantiles.html>
- [22] Prometheus, “Histograms and summaries.” <https://prometheus.io/docs/practices/histograms/>
- [23] R. B. Miller, “Response time in man-computer conversational transactions,” Proc. AFIPS Fall Joint Computer Conference, vol. 33, pp. 267–277, 1968.
- [24] Groq, “GroqCloud API Documentation” and “Rate Limits,” 2026. <https://console.groq.com/docs/rate-limits>